

AI Agent Integration Documentation

Pulse Pariraksha AI Assistant

1. Introduction

This project implements an AI-powered chatbot assistant that provides information about the Pulse Pariraksha preventive health checkup package. The system combines rule-based intent detection, a Firestore knowledge base, and generative AI responses to deliver accurate and efficient answers.

2. System Architecture

User → React Chat Widget → Backend API (/api/chat) → Intent Detection → Firestore Knowledge Base → Gemini AI → Response to User.

3. Technology Stack

Frontend: React

Backend: Node.js + Express

Database: Firebase Firestore

AI Model: Gemini

Security: Rate limiting and environment variables.

4. Frontend Integration

The frontend chat widget captures user input, displays messages, and sends requests to the backend API. It manages UI elements such as typing indicators, quick replies, and message history.

5. Backend Server

The backend server built with Express handles chat requests, performs intent detection, retrieves knowledge from Firestore, and calls the Gemini AI model when necessary.

6. Intent Detection

The backend checks user messages for keywords related to price, fasting, duration, tests, or booking. If an intent is detected, the server can return a direct response without calling the AI model.

7. Firestore Knowledge Base

Firestore stores structured health package information such as price, fasting rules, duration, and included tests. The backend retrieves this data and injects it into AI prompts.

8. AI Fallback Mechanism

If the question does not match known intents, the backend sends the question along with system instructions and knowledge base information to the Gemini AI model to generate a response.

9. Prompt Construction

The AI prompt includes system instructions, knowledge base content, and the user's question. This ensures that the AI provides accurate responses without hallucinating information.

10. Conversation Memory

The system optionally includes recent chat history to maintain conversational context between the user and the assistant.

11. Error Handling

All AI requests are wrapped in try/catch blocks. If an error occurs, the system logs the issue and returns a fallback message to the user.

12. Security

API keys are stored securely in environment variables. Rate limiting prevents excessive API usage or abuse.

13. Performance Optimization

Intent detection allows the chatbot to answer many questions instantly without invoking the AI model, reducing cost and improving speed.

14. Example Interaction Flow

Example: User asks 'Fast cheyyano?' → Intent detection identifies 'fasting' → Firestore provides fasting rule → Response returned instantly.

15. Future Improvements

Possible enhancements include vector-based semantic search, multi-document knowledge bases, conversation memory storage, and tool-based AI agents for automated booking workflows.